Mysten Republic

Security Assessment

October 28th, 2024 — Prepared by OtterSec

Robert Chen r@osec.io

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-RPL-ADV-00 | Repeated Invocation Resulting in Excessive Claims 6

OS-RPL-ADV-01 | Risk of Compromising Snapshot Integrity 7

OS-RPL-ADV-02 | Possible Overflow Due to Exceeding the Type Limit 8

OS-RPL-ADV-03 | Failure to Update Default Group 9

General Findings **10**

OS-RPL-SUG-00 | Unauthorized Privileges Due to Group ID Reassignment 11

OS-RPL-SUG-01 | Code Refactoring 12

OS-RPL-SUG-02 | Code Maturity 13

Appendices

Vulnerability Rating Scale **15**

Procedure **16**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Mysten engaged OtterSec to assess the `republic` program. This assessment was conducted between October 11th and October 21st, 2024. A follow-up review was performed between April 7th and April 13th, 2025. and For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 7 findings throughout this audit engagement.

In particular, we identified a vulnerability in the claim functionality concerning a lack of a check to ensure that the cumulative amount claimed, including previous claims, does not exceed the total unlocked tokens. This enables users to repeatedly claim more than the intended unlocked balance (OS-RPL-ADV-00). Additionally, during the snapshot process, if tokens included in the snapshot are combined with those not part of it, the integrity of the snapshot is compromised (OS-RPL-ADV-01). Furthermore, we highlighted the possibility of an overflow in multiplication and division operations if the values exceed the maximum limit representable by a `u64` type (OS-RPL-ADV-02).

We also made a recommendation to modify the codebase for enhanced functionality, efficiency, and maintainability (OS-RPL-SUG-01) and suggested ensuring adherence to coding best practices (OS-RPL-SUG-02).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/MystenLabs/security-token. This audit was performed against ff7d0a0. A follow-up review was performed against 13e91eb.

A brief description of the program is as follows:

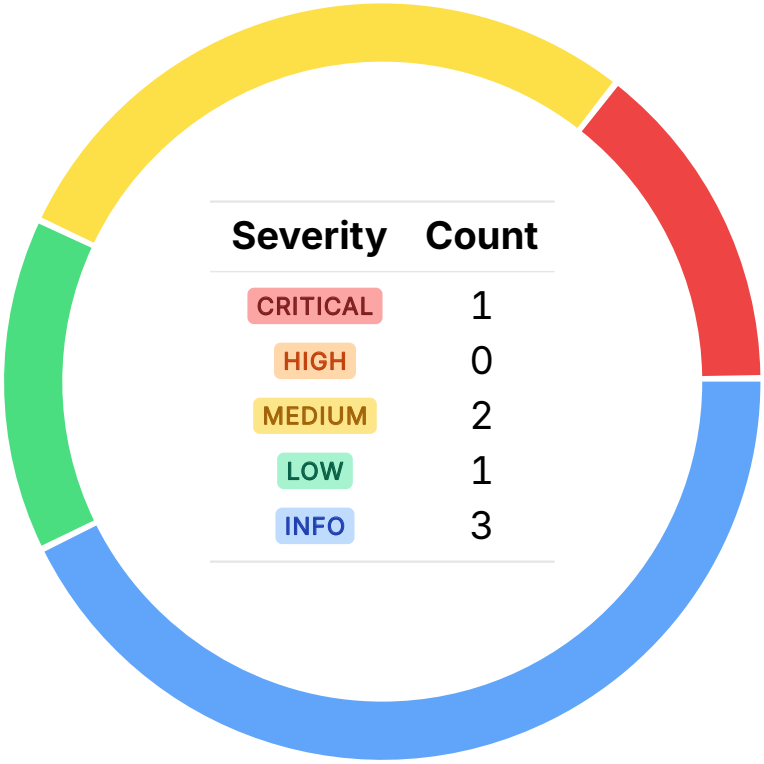
Name	Description
republic	A treasury management system that enforces strict access control and permissions through roles and abilities, within the Sui environment.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 7 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	1
HIGH	0
MEDIUM	2
LOW	1
INFO	3

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-RPL-ADV-00	CRITICAL	RESOLVED	`claim` lacks a check to ensure that the cumulative amount claimed, including previous claims, does not exceed the total unlocked tokens, allowing users to repeatedly claim more than the intended unlocked balance.
OS-RPL-ADV-01	MEDIUM	RESOLVED	During the snapshot process, if tokens included in the snapshot are combined with those not part of it, the integrity of the snapshot is compromised.
OS-RPL-ADV-02	MEDIUM	RESOLVED	In `calculate_available_dividends` and `calculate_unlocked`, the multiplication and division operations may overflow if the values exceed the maximum limit representable by a `u64` type.
OS-RPL-ADV-03	LOW	RESOLVED	`remove_address_from_group` updates the `address_info` for `addr` to set it to the `DEFAULT_GROUP`, but it fails to update the `DEFAULT_GROUP` entry in `Group.holders`.

© 2024 Otter Audits LLC. All Rights Reserved.

Repeated Invocation Resulting in Excessive Claims CRITICAL OS-RPL-ADV-00

Description

In `timelock::claim`, the module does not fully account for the cumulative claims an owner may make on the unlocked tokens. Without this additional check, a user may exploit the function to repeatedly claim the maximum allowable unlocked balance, potentially exceeding the actual unlocked amount over multiple calls. The function calculates `unlocked_tokens`, which is the total amount of tokens unlocked up to the current timestamp, based on the timelock's release schedule.

>_ *treasury_abilities/timelock/timelock.move* RUST

```rust
public fun claim<T>(
    self: &mut Timelock<T>,
    amount: Option<u64>,
    clock: &Clock,
    ctx: &mut TxContext,
): SplitRequest<T> {
    [...]
    let amount = amount.destroy_or!(unlocked_tokens);
    assert!(amount <= unlocked_tokens, ENotEnoughBalanceUnlocked);
    self.tokens_transferred = self.tokens_transferred + amount;
    let coin = self.left_balance.split(amount).into_coin(ctx);
    let (token, request) = shared_token::from_coin(coin, ctx);
    token.share();
}
```

However, each call to claim only checks that the currently requested amount is within the currently unlocked tokens, which renders the function vulnerable to repeated withdrawals. While `self.tokens_transferred` increments by the claimed amount with each call, there is no check to ensure that `amount + self.tokens_transferred` does not exceed the cumulative `unlocked_tokens`. This oversight allows the owner to call `claim` multiple times within the same unlock period, requesting the maximum unlocked amount each time without accounting for previously claimed tokens.

Remediation

Verify that the cumulative amount claimed does not exceed the total unlocked tokens (`amount + self.tokens_transferred <= unlocked_tokens`).

Patch

Resolved in fe96359.

© 2024 Otter Audits LLC. All Rights Reserved.

Risk of Compromising Snapshot Integrity MEDIUM OS-RPL-ADV-01

Description

The vulnerability concerns the integrity of the snapshot process. A successful snapshot relies on accurately capturing the total supply of tokens and distinguishing between those that are counted (unlocked) and those that are not (locked). `join` allows two different tokens to be merged into one, potentially altering the balances and total supply of tokens mid-snapshot. If tokens that are part of the snapshot join with those that are, `total_supply` will no longer be equal to `unlocked_sum + locked_sum`.

Remediation

Employ a locking mechanism to block `shared_token::join` during the snapshot process.

Patch

Resolved in f6fa43b.

© 2024 Otter Audits LLC. All Rights Reserved.

Possible Overflow Due to Exceeding the Type Limit MEDIUM OS-RPL-ADV-02

Description

There is a potential overflow risk in `dividends::calculate_available_dividends` during the calculation of dividends for a particular address. Specifically, when multiplying two large `u64` values, such as `self.total_funds` and `snapshot.address_balance(addr)`, the result may exceed the maximum value for `u64`, resulting in an overflow.

>_ *treasury_abilities/pause/dividends.move* RUST

```rust
public fun calculate_available_dividends<T, U>(
    self: &Dividends<U>,
    snapshot: &Snapshot<T>,
    addr: address,
): u64 {
    [...]
    // prettier-ignore
    let addr_total_dividends =
        (self.total_funds * snapshot.address_balance(addr)) / unlocked_supply_t;
    addr_total_dividends - already_claimed
}
```

Similarly, `shecdule_config::calculate_unlocked` contains multiple multiplications and divisions involving `u64` values, which may result in an overflow when handling large amounts or time periods. For example, while calculating `unlocked`, `full_amount`, and `initial_release_portion_in_bps` may be large values, and multiplying them in `u64` space may exceed the `u64` limit.

Remediation

Ensure the values involved in the multiplication and division are converted to `u128` before performing the arithmetic operations. Moving to `u128` will provide a much larger range, removing the risk of overflow.

Patch

Resolved in a393844.

© 2024 Otter Audits LLC. All Rights Reserved.

Failure to Update Default Group LOW OS-RPL-ADV-03

Description

Within `remove_address_from_group` in `group_holder_config`, the specified address is removed from its current group (old group), and the `address_info` for the address is updated, setting its `group_id` to `DEFAULT_GROUP`. However, it does not add the address to the holders list of the default group in `config.groups`. By setting `address_info.group_id` to `DEFAULT_GROUP`, the function intends to reassign the address to the default group.

>_ rules/group_holder_config/group_holder_config.move RUST

```rust
/// Will set the address to the DEFAULT_GROUP.
public fun remove_address_from_group<T>(
    policy: &mut TokenPolicy<T>,
    cap: &TokenPolicyCap<T>,
    addr: address,
) {
    [...]
    // Remove address from old group
    let old_group = address_info.group_id;
    let group = &mut config.groups[&old_group];
    group.remove_holder_address(address_info.holder_id, addr);
    // Update addresses Table
    address_info.group_id = DEFAULT_GROUP;
    // If the below assertion fails, it means that the state of the group-holderconfig is
        ↪ corrupted.
    let holder_addresses = config.holders[address_info.holder_id];
    assert!(holder_addresses.contains(&addr), EImplementationError);
}
```

However, without updating the `DEFAULT_GROUP` in `Group.holders`, the reassignment will be incomplete. This will result in an inconsistency, as the address will appear to belong to the default group based on `address_info`, but it will not be listed as a holder within the actual `DEFAULT_GROUP` in `config.groups`.

Remediation

Ensure that the address is explicitly added to the holders of the `DEFAULT_GROUP`.

Patch

Resolved in f960290.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-RPL-SUG-00	An admin may arbitrarily set group IDs, allowing for a new group to take the place of a removed group in an allowed group pair, resulting in access to unauthorized privileges.
OS-RPL-SUG-01	Recommendation for modifying the codebase for improved functionality, efficiency, and maintainability.
OS-RPL-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2024 Otter Audits LLC. All Rights Reserved.

Unauthorized Privileges Due to Group ID Reassignment OS-RPL-SUG-00

Description

In the current implementation, the management of group IDs is insufficiently controlled, especially regarding the administrator's ability to create and assign arbitrary group IDs. For instance, if groups `G0` and `G1` exist and `allow_group_pair` is created to call a pair of groups `(G0, G1)`, and the group `G0` is removed, the ID associated with that group becomes available for re-utilization.

>_ *treasury_abilities/pause/dividends.move* RUST

```rust
public fun add_group<T>(
    auth: &Auth<T>,
    policy: &mut TokenPolicy<T>,
    group_id: u64,
    group_name: String,
    max_holders: Option<u64>,
    ctx: &mut TxContext,
) {
    assert!(auth.has_ability<T, EditGroupsAbility>(), ENotAllowedToEditGroups);
    policy.add_group(auth.policy_cap(), group_id, group_name, max_holders, ctx);
}
```

Consequently, it is possible to add a new group (now with ID `G0`) after the original `G0` is removed. As a result, the new group will be treated as a member of the allowed group pair `(G0, G1)`. Thus, it may interact with `G1` without any checks on its legitimacy or authority, allowing unauthorized transfers of tokens or privileges that were intended to be restricted to the original `G0` group. The system's integrity relies on the assumption that group IDs correspond to trusted entities.

By allowing a new group to adopt the ID of a previously removed group, the system undermines its own trust model. Any operations relying on group pairs may fail to enforce the intended security boundaries. The same vulnerability applies to `lock_group_until_timestamp_rule`.

Remediation

Ensure that once a group ID is removed, it is not possible to immediately create another group with a similar index.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring

OS-RPL-SUG-01

Description

1. Modify `force_transfer_amount` so that the `ForceTransferred` event is only emitted once rather than once in the internal call to `force_transfer` and then again in `force_transfer_amount`. Additionally, we recommend emitting events in the `accepted_currencies` and `group_holder_config` modules to improve observability.
2. Update `add_holder_address_internal` to utilize the `group_id` parameter instead of the hardcoded `DEFAULT_GROUP`, so that the function is more flexible, allowing it to add addresses to any specified group, instead of just the default group.

   >_ *sources/rules/group_holder_config/group_holder_config.move* RUST

   ```rust
   fun add_holder_address_internal<T>(
       self: &mut Config,
       holder_id: u64,
       addr: address,
       group_id: u64,
   ) {
       event::emit(HolderAddressAdded<T> { holder_id: holder_id, addr });
       self.holders[holder_id].insert(addr);
       self.addresses.add(addr, AddressInfo { holder_id, group_id });
       self.groups[&default_group()].add_holder_address(holder_id, addr);
   }
   ```

3. In `shared_token`, both `action_request` and `request_inner` perform the same function. Thus, it would be appropriate to keep only one of them.

Remediation

Implement the above-mentioned refactors.

Patch

1. Issue #1 resolved in 01504c5.
2. Issue #2 resolved in b3a4edf.
3. Issue #3 resolved in f960290.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-RPL-SUG-02

Description

1. Within `dividends`, there may be potential rounding discrepancies, especially after all users have claimed, as most dividend calculations involve proportional distribution, which may result in small leftover amounts due to rounding.
2. In `freeze_rule::add`, when a new `FreezeRule` configuration is created and added to the `TokenPolicy` (in the `else` block), it currently initializes the configuration with an empty `VecSet` for `frozen_addresses`, rather than adding the provided addresses from `frozen_addresses` into this new configuration. This implies that if this is the first time the `FreezeRule` is added to the policy, no addresses from the provided `frozen_addresses` vector will actually be frozen at this point.

   >_ *sources/rules/freeze_rule.move* RUST

   ```rust
   public fun add<T>(
       policy: &mut TokenPolicy<T>,
       cap: &TokenPolicyCap<T>,
       action: String,
       frozen_addresses: vector<address>,
       ctx: &mut TxContext,
   ) {
       token::add_rule_for_action<T, FreezeRule>(policy, cap, action, ctx);
       if (policy.has_rule_config_with_type<T, FreezeRule, Config>()) {
           frozen_addresses.do!(|addr| freeze_address(policy, cap, addr, ctx));
       } else {
           token::add_rule_config(
               FreezeRule {},
               policy,
               cap,
               Config { frozen_addresses: vec_set::empty() },
               ctx,
           );
       };
   }
   ```

3. Currently, there is no role check in `snapshot::cancel_snapshot` to ensure that only authorized users with specific roles may cancel an ongoing snapshot. In particular, since the function resumes paused treasury and token operations, proper access control is necessary to deter unauthorized cancellation.
4. It is recommended to utilize `u128` for certain calculations to prevent overflows, as it supports larger value ranges and ensures safe arithmetic operations.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

1. Ensure the funder retrieves the remaining balance in `dividends` after all users have claimed.
2. Insert `frozen_addresses` into the configuration in `freeze_rule::add`.
3. Implement access control to verify if the caller has the required role to cancel the snapshot.
4. Utilize `u128` in calculation.

Patch

1. Issue #1 resolved in 9980fe5.
2. Issue #2 resolved in 8011e96.
3. Issue #3 resolved in f960290.

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.